

Latent action space facilitates transfer learning in Reinforcement Learning agents

Jean Fregeville
EPFL, Centrale Lille

Farhan Ali
EPFL

Vasileios Gkikas
EPFL

Abstract—Numerous papers in the literature demonstrate the effectiveness of Curriculum Learning. We hereby explore the effectiveness of curricula for solving Wordle and demonstrate how latent action spaces can enable transitions to more difficult environments without architectural changes while preserving agents’ performance at the game.

I. INTRODUCTION

Training reinforcement learning algorithms in environments with a large discrete action space often poses significant convergence challenges. In the case of the game Wordle[1], directly exposing an agent to the entire dictionary creates a vast search space. This high variance in reward signals slows down learning. To overcome this constraint, we investigate the impact of Curriculum Learning (CL) via a progressive “warm-up” approach. By introducing a latent action space, the agent learns to master easier tasks in action subspaces without requiring any changes in its internal architecture. Our contributions are listed below:

- We developed a lightweight framework that can run on a consumer-grade CPU for training an RL agent to solve Wordle based on [2] and [3].
- We created a curriculum partition of 3 subsets of words based on their difficulty.
- We compared the agents’ performance when trained without any constraint versus when trained on gradually expanding subsets of words, in one case randomized and in another according to the aforementioned curriculum.

We have released the source code on GitHub [4].

II. BACKGROUND

A. Reinforcement Learning

Reinforcement learning (RL) is concerned with how an intelligent agent should take actions in a dynamic environment in order to maximize a reward signal [5]. Here, “intelligent agent” refers to an entity that perceives its environment, takes actions autonomously to achieve goals, and may improve its performance.

This process is typically modeled (see [6]) as a Markov Decision Process (MDP), defined by the tuple (S, A, P, R, γ) , where: S represents the state space, A is the action space, P represents the state transition probability measure, i.e. the agent’s expectations of moving from one configuration to another, R is the reward function, and γ is a discount factor.

The agent’s objective is to learn an optimal policy function $\pi^* : S \rightarrow A$ such that π^* maximizes the expected reward.

B. Curriculum Learning

Curriculum Learning (CL), introduced by Bengio et al. in [7], is a training strategy inspired by the human education system.

By initially training the agent on easier (according to some natural metric) training data for the target task, the model can quickly discover a basic policy that performs well. As training progresses, the training data difficulty is incrementally increased and eventually matches the complete distribution of the target task.

III. RELATED WORK

There have been previous attempts at utilizing RL strategies to create good RL solvers, such as [8] and [3]. [9] was one of these previous attempts that utilized the Advantage Actor-Critic (A2C) Model, with an implementation present in [2].

Our work studies the feasibility of improving the above mentioned A2C model for Wordle using curriculum learning to optimize the training period, which was first introduced in [7]. We utilise a lot of insights from other papers that study wordle solving strategies such as [10], [11] and [12].

We also try to obtain an empirical validation of warm-starting policies via abstract latent space exploration in [13], which is another type of optimization in the training phase, here applying it to large discrete action spaces.

IV. DATASET

We could not find official datasets used for the actual game of Wordle. The full dataset of available guesses employed in this study comes from [2]. A smaller dataset from [3], containing the actual possible answers, was also used.

V. METHODOLOGY

In this section, we formalize the neural architecture, reward scheme, and parameters of our baseline Reinforcement Learning model. The agent interacts with the Wordle environment using an Advantage Actor-Critic architecture.

A. State Space

The state $s \in S$ provided to the agent at each step is structured as a vector $s \in \mathbb{R}^{391}$. The first coordinate keeps track of the number of attempts left, and the remaining 390 (5 positions $\times$ 26 letters $\times$ 3 feedback indices) are initialized at 0 and updated after each turn to reflect the feedback provided by the game.

B. Rewards

The agent obtains a reward of +2, if it finds a letter in the correct position (Green) and +1, if it finds a letter in a wrong position (Yellow). If it guesses the word, it also obtains a bonus of +10 for every attempt it had left. Failure to find the correct word results in a penalty of -50. The intermediate rewards help navigate the agent towards the correct solution, while the final rewards serve the purpose of discouraging exploitation of the reward system.

C. Neural Network Architectures

Our Actor-Critic framework utilizes two separate neural networks. Both networks share the same 391-dimensional input layer and a hidden layer mapping the input to a 256-dimensional vector with a ReLU activation function, and a linear output layer of dimension 130 for the Actor and 1 for the Critic. Herein, the Critic estimates a value function $V(s)$ for each state, and the actor computes an Advantage function $A(s, a)$ [14] for each state s and action a .

We define the Temporal Difference (TD) error as $\delta = R + \gamma V(s') - V(s)$, where R is the reward obtained after taking an action a and transitioning from state s to s' , and use δ as a proxy for the advantage. The actor's loss (for a single state) is then defined as $\frac{1}{B} \sum_{s \in S} (-\log(\pi_\theta(a | s)) \cdot \delta' - \beta H(\pi(\cdot | s)))$, where B is the batch size, δ' is δ normalized over B , and H is the entropy of the distribution. The critic's loss is defined as the mean δ^2 over all states visited in a batch.

D. From Latent Action Space to Action

To map the actor's output to words, we utilize a One-Hot Encoding (OHE) per letter position to obtain matrix $M \in \mathbb{R}^{130 \times |W|}$, where W denotes the set of words the actor can choose from at any point. Each word is therefore mapped to its corresponding embedding $e \in \mathbb{R}^{130}$, and $\|e\|_2 = \sqrt{5}$. By calculating $a^T e$, we obtain a score for each available word. Since all possible embeddings e have the same norm, this is analogous to calculating the cosine similarity between a and all e . The scores are passed as logits to a categorical distribution, from which an action is sampled. This architecture enables a seamless transition from higher-dimensional action spaces and builds upon established frameworks for large discrete action spaces, where policies operate over continuous latent representations before mapping to discrete choices [15].

E. Training Configuration and Hyperparameters

We formulated a multi-stage training framework, which we designate as the "warm-up" method. This technique sequentially expands the active training dictionary over time, allowing the agent to consolidate its tracking and elimination logic on simpler tasks before scaling up to higher operational difficulties.

The training pipelines for both the naive (no CL) and the warm-up configurations of the RL agents are structured around an episodic batch-updating routine. Both networks employ an Adam optimizer to update their learned parameters.

The following hyperparameter configuration was used:

- **Learning Rate:** Set to 0.001 for both the Actor and the Critic networks, ensuring steady gradient descent steps.

- **Batch Size:** Configured to 4 complete game trajectories to ensure updates are neither noisy nor infrequent enough to inhibit learning.
- **Discount Factor γ for the value function:** Set to 0.99 to encourage pursuit of long-term rewards.
- **Entropy Regularization Coefficient β :** Set to 0.1. This term encourages vocabulary exploration during early training phases and when increasing the action space, preventing the policy from collapsing. [16]

All models are evaluated over 500,000 training episodes.

F. Action space expansion

Initially, the agent can choose from a subset of 20 words, which is also the subset of all possible answers W . After 2000 episodes, this set is expanded to 100 words, then to 500 after 20000 total episodes. After 40000 complete episodes, the restrictions to the agent's actions and the possible answers are lifted. In the case of the hand-crafted curriculum, each increase in size corresponds to an increase in difficulty: the agent starts with a subset of "easy" words, then the set gets expanded to include "medium" and later "hard" words. The subset cardinalities and expansion timepoints are the same in the two variations.

VI. CURRICULUM DEFINITION

Here we establish a quantifiable metric to evaluate the difficulty of each Wordle target word. From an information theoretic standpoint, the difficulty of an RL agent being able to guess a hidden target word heavily relies on the frequency of its letters and the presence of duplicates [11].

A. Letter Frequency

Words composed of highly frequent letters are considered easier [17]. Let $U(w)$ be the set of unique characters in the word w , and $P(c)$ be the probability of occurrence of character c in the dataset. We define the frequency score as:

$$S_{freq}(w) = \sum_{c \in U(w)} P(c) \quad (1)$$

B. Duplicate Letters (Penalty)

Words containing duplicate letters (e.g., "MUMMY") are harder for RL agents [12]. Duplicates reduce the number of unique alphabet letters scanned per guess, effectively limiting environment exploration. We therefore applied a penalty P_{dup} for every duplicate letter in the word.

$$P_{dup}(w) = (|w| - |U(w)|) \times \alpha \quad (2)$$

where $|w|$ is the total length of the word (5 in Wordle) and α is a tunable penalty weight, here set to 0.15.

C. Difficulty Metric

The difficulty score of a word is defined as :

$$Score(w) = S_{freq}(w) - P_{dup}(w) \quad (3)$$

Words are sorted in descending order based on this score. This ordered list is then split into three equally sized subsets (Easy, Medium, Hard) to form the curriculum, mirroring recent processes used to classify Wordle target distributions [18].

VII. RESULTS

The naive model without any CL exhibited poor performance during the initial training phase, requiring ~ 150000 episodes to obtain a positive average reward and a win-rate above 0.5 (Fig. 1 and 2). On the contrary, both CL models exhibited strong performance early in the training stage, mastering the easier tasks after tens of thousands of episodes. At the same time, the second 5-fold dataset expansions result in a 3-fold win-rate decrease, indicating the model’s ability to generalize to and guess unseen words. This is also corroborated by the last drop in performance: during the last stage, the model’s potential actions are increased from 500 to above 12000, yet the models’ performance matches that of the naive agent after 80000 iterations.

As for the curriculum learning approach, a randomized curriculum appears to perform just as well as a hand-crafted curriculum that also jumps in difficulty as training progresses. This indicates that the fundamental strategy of an RL agent trying to solve the game does not change significantly based on the measurable difficulty of a target word. While the distinction between “easy”, “medium” and “hard” words definitely exists, the jump in difficulty levels turned out to lower than initially expected, and the most important factor for understanding the game was mastery of a subset of words.

Despite the early advantages of the CL models, after ~ 300000 episodes the naive model manages to reach their performance, after which point all models perform largely similar across the different tested metrics. This is not unexpected: there exist multiple implementations for solving Wordle without the need of a curriculum as defined here [2][3].

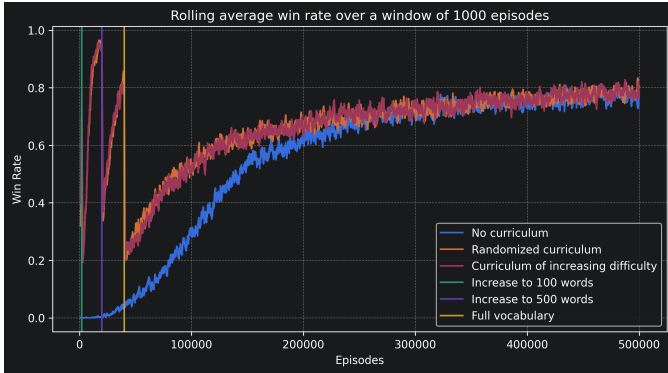


Fig. 1: Win rate evolution over 500,000 training episodes, showing the comparison between Naive (no CL) and CL strategies.

VIII. DISCUSSION

Curriculum learning was introduced as a means of accelerating training in Machine Learning applications, by gradually introducing a larger number of more complex topics. In this work, we evaluated transferability of learning across both sophisticated and randomized curricula. While the asymptotic performance of all agents converged across all measured metrics, the CL agents exhibited stronger performance for a significant number of iterations, indicating their ability

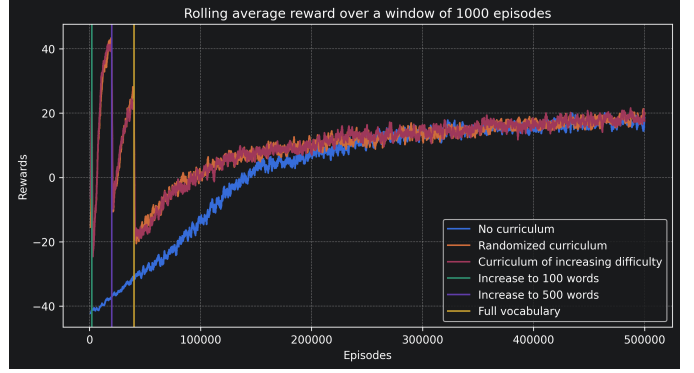


Fig. 2: Reward history comparison under the same training conditions as Fig. 1. The reward curves mimic the winning rate curves, indicating that the chosen reward scheme was appropriate for the task.

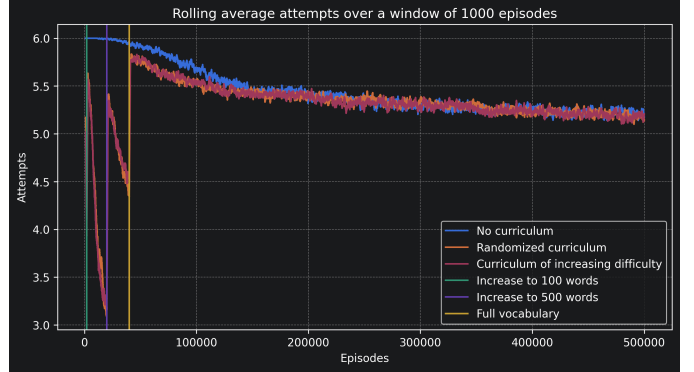


Fig. 3: Attempt history comparison under the same training conditions as Fig1. As the models become more capable, the number of required attempts drops

to generalize well to new tasks. While we were unable to accurately measure the training time of the different models, our experiments also showed that the curriculum learning phase required $\sim 2 - 4$ times less time for the same number of episodes compared to the agent that was exposed to the full sets from the beginning. In light of this, we believe similar methods can accelerate performance across different applications.

IX. FUTURE WORK

Our Actor and Critic models consisted of a small number of layers to ensure low backpropagation times and to facilitate transfer of knowledge by avoiding overfitting. Future work could ablate model complexity to evaluate how well different networks can generalize when introduced to higher-dimensional action spaces. Experiments quantifying the computational efficiency gains (e.g. in FLOPs or seconds) would also be a beneficial way forward.

A third possible direction would be studying if our proposed method could yield similar improvements to other architectures, such as encoder-decoder models. For example, after training of an encoder, the decoder could be initially trained on text containing subsets of the embeddings, such as texts from specific languages or genres, and gradually be exposed to more data.

REFERENCES

- [1] T. N. Y. Times, “Wordle,” <https://www.nytimes.com/games/wordle/index.html>, 2022, accessed: 2026-04-20.
- [2] harsh788, “wordle : A deep reinforcement learning approach,” Available online, 2024. [Online]. Available: <https://github.com/harsh788/woRdLe/tree/7c9a85ecf95ec933240f890bc4c8adce65e1e28>
- [3] Voorhs, “Deep q networks for playing wordle,” Available online, 2023. [Online]. Available: <https://github.com/voorhs/wordle-rl/tree/88e7dd1d41d5e81b469e1aae461e4b615d492fbc>
- [4] vgkikas, “Curriculum learning for wordle rl agents via latent action space exploitation,” Available online, 2026. [Online]. Available: <https://github.com/vgkikas/woRdLe>
- [5] Wikipedia contributors, “Reinforcement learning — Wikipedia, the free encyclopedia,” 2026, online; accessed 20-April-2026. [Online]. Available: https://en.wikipedia.org/wiki/Reinforcement_learning
- [6] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018, accessed: 2026-04-20. [Online]. Available: <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>
- [7] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proceedings of the 26th annual international conference on machine learning*, 2009, p. 1, accessed: 2026-04-20. [Online]. Available: https://www.researchgate.net/publication/221344862_Curriculum_learning
- [8] S. Bhambri, A. Bhattacharjee, and D. Bertsekas, “Reinforcement learning methods for wordle: A pomdp/adaptive control approach,” 2022. [Online]. Available: <https://arxiv.org/abs/2211.10298>
- [9] Anonymous, “Solving wordle using actor-critic reinforcement learning,” in *Proceedings of the IEEE Conference on Artificial Intelligence and Games*, November 2025, explores Vanilla AC and A2C combined with curriculum training to scale from tiny vocabularies to the entire Wordle lexicon. [Online]. Available: <https://ieeexplore.ieee.org/document/11294192>
- [10] B. J. Anderson and J. G. Meyer, “Finding the optimal human strategy for wordle using maximum correct letter probabilities and reinforcement learning,” *arXiv preprint arXiv:2202.00557*, 2022. [Online]. Available: <https://arxiv.org/abs/2202.00557>
- [11] A. Shrestha and et al., “Using information theory to play wordle as optimally as possible,” *National High School Journal of Science*, 2024, section: ‘Maximizing Information Gain (Entropy)’. [Online]. Available: <https://nhsjs.com/2024/using-information-theory-to-play-wordle-as-optimally-as-possible/>
- [12] Y. Huang and et al., “A study on the development of wordle based on machine learning,” in *2023 IEEE 3rd International Conference on Data Science and Computer Application (ICDSCA)*. IEEE, 2023, pp. 124–129, section III-B: ‘Clustering Algorithm for Difficulty Assessment’. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10393779&casa_token=aZauwEHewAAAAA:b1VhgsH3maMXILnnJLSb71-dX58w6etxItfAgipTg27akNXJORMls_t3fEFXNOCaJ_9H4f15Gw&tag=1
- [13] S. S. K and A. Hazra, “Spaars: Safer rl policy alignment through abstract exploration and refined exploitation of action space,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.09378>
- [14] G. Wulfram, “Cs-479: Learning in neural networks,” EPFL Course, 2026, course material.
- [15] Y. Chandak, G. Theodorou, J. E. Kostas, S. M. Jordan, and P. S. Thomas, “Learning action representations for reinforcement learning,” *CoRR*, vol. abs/1902.00183, 2019. [Online]. Available: <http://arxiv.org/abs/1902.00183>
- [16] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” 2016. [Online]. Available: <https://arxiv.org/abs/1602.01783>
- [17] C. Tsang, “Demystifying wordle: A crash course in information theory,” *Science Focus*, 2022, section: ‘Information Gain and Entropy’. [Online]. Available: <https://sciencefocus.hkust.edu.hk/demystifying-wordle-a-crash-course-in-information-theory>
- [18] H. Bruseantsov, *Application of Reinforcement Learning Tools for Solving Puzzle Games*, 10 2025, pp. 23–41.
- [19] K. Chusap, C. Murphy, C. Hess, S. D. Kadaru, R. Buccapatnam, and C. Liu, “RL- and llm-based ai solvers for the game of wordle/fibble,” in *2025 IEEE Conference on Games (CoG)*, 2025, pp. 1–8.
- [20] G. Dulac-Arnold, R. Evans, H. van Hasselt, P. Sunehag, T. Lillicrap, J. Hunt, T. Mann, T. Weber, T. Degris, and B. Coppin, “Deep reinforcement learning in large discrete action spaces,” 2016. [Online]. Available: <https://arxiv.org/abs/1512.07679>
- [21] J. Zhang, X. Wang, and Y. Li, “Discretizing continuous action space with unimodal probability distributions for on-policy reinforcement learning,” *arXiv preprint arXiv:2408.00309*, 2024, discusses the theoretical pitfalls of policy gradient estimators when discrete action spaces explode, providing a foundational backbone for structural action representation. [Online]. Available: <https://arxiv.org/abs/2408.00309>
- [22] E. Zelikman, E. Schmerling, M. Pavone, and N. D. Goodman, “Learning to act without actions: Latent action policies from unlabeled data,” *arXiv preprint arXiv:2312.10812*, 2024. [Online]. Available: <https://arxiv.org/abs/2312.10812>
- [23] J. D. Hunter, “Matplotlib: A 2d graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [24] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: <https://doi.org/10.1038/s41586-020-2649-2>
- [25] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhrsch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala, “PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation,” in *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, Apr. 2024. [Online]. Available: <https://docs.pytorch.org/assets/pytorch2-2.pdf>